Module: analytics engine and risk scoring

English version of ../../modules/analytics-engine.md.

1. Purpose and boundaries

Computes the service-renewal risk and the revenue at risk from a data snapshot. It works as a pure computation over the domain (`backend/app/domain/*`, `core.py`). It does not fetch data from the network, does not decide what to show the user and holds no secrets. Risk is an explainable heuristic, **not** a churn probability.

2. Use scenarios

- Building the overview: aggregates over accounts, services and revenue.
- Computing the risk of a service with the contribution of every factor.
- Computing revenue at risk (the price of the nearest renewal × the risk score).
- Replacing the snapshot and recomputing: numbers change consistently, provenance is updated.
- Failure: the domain is unavailable → `UnavailableAnalytics`, the API answers 503 instead of returning emptiness.

3. Contract

`/api/v1/analytics/*`, `/api/v1/customers/*` — in `backend/openapi.json`. The internal contract is `backend/app/domain/module.py` (`create()`), replaceable through `AEZA_ANALYTICS_FACTORY` (needed for tests and alternative implementations).

4. Data

Owns the aggregates and the scores; reads the snapshot (accounts, services, transactions). Money — `domain/money.py`, integer minor units. Provenance — `domain/provenance.py`: every number knows its source.

5. Access rules

Reading the analytics requires authentication (JWT from `backend/auth`); snapshot ingestion additionally requires `AEZA_SOURCE_INGEST_TOKEN`. Refusals are 401/403 without disclosing data.

6. Dependencies

Outgoing: the standard library and its own domain only (the network is not used). Incoming: `backend/app/api/*`, AI tools (`backend/app/ai/tools/analytics.py`).

7. Configuration

`AEZA_AS_OF` (pinned calculation date), `AEZA_ANALYTICS_FACTORY`, `AEZA_RECALCULATE_SECONDS`, `AEZA_SEED_DEMO_DATA`. A wrong `AEZA_ANALYTICS_FACTORY` breaks the start with a clear error — better than silently substituting the calculations.

8. Build and debug

There is no separate binary. Locally:

```
python -m pytest backend/tests/test_domain.py backend/tests/test_api.py -q
```

9. Tests

`backend/tests/test_domain.py` (rules and money), `test_api.py` (routes), `test_independent_acceptance.py` (independent scenario checks), `test_actions.py`, `test_deck_values.py` (the numbers in the presentation match the API). The main guarantee "the score is explainable" is the factor-contribution check.

10. Observability

Latency metrics per route (`/metrics`), snapshot provenance, 503 codes when the domain is unavailable. Operator action: compare the snapshot date with `AEZA_AS_OF`.

11. Limitations and readiness

- No labelled outcomes → no calibration and no ROC metrics; the risk cannot be called a probability.
- No before/after scenario comparison in a single response.
- Readiness: the numbers in the API and the presentation match (`test_deck_values.py`); replacing the snapshot does not break aggregate consistency.